

# JavaScript Theory Assignment

## 1. JavaScript Introduction

Question 1: What is JavaScript? Explain the role of JavaScript in web development. JavaScript is a high-level, interpreted programming language that is primarily used to make web pages interactive and dynamic. It is one of the core technologies of web development, along with HTML and CSS.

The role of JavaScript in web development includes:

- Adding interactivity (e.g., buttons, forms, animations)
- Manipulating HTML and CSS dynamically
- Handling user inputs and events
- Communicating with servers using APIs (AJAX, Fetch)

Without JavaScript, web pages would remain static and less engaging.

Question 2: How is JavaScript different from other programming languages like Python or Java?

JavaScript differs from Python and Java in several ways:

- Execution Environment: JavaScript runs in browsers, while Python and Java run on servers or standalone systems.
- Typing System: JavaScript is loosely typed, whereas Java is strongly typed.
- Compilation: JavaScript is interpreted, while Java is compiled.
- Usage: JavaScript is mainly used for web development, while Python is used in data science, AI, and backend, and Java is used in enterprise applications.

Question 3: Discuss the use of <script> tag in HTML. How can you link an external JavaScript file?

The <script> tag is used to include JavaScript code in an HTML document.

It can be used in two ways:

<script>

1. Internal JavaScript

```
console.log("Hello World");
```

</script>

2. External JavaScript

```
<script src="script.js"></script>
```

## 2. Variables and Data Types

Question 1: What are variables in JavaScript? How do you declare them?

Variables are containers used to store data values.

JavaScript provides three ways to declare variables:

var (old method, function-scoped)

let (block-scoped, modern)

const (constant, cannot be reassigned)

Example:

```
var a = 10;
```

```
let b = 20;
```

```
const c = 30;
```

Question 2: Explain different data types in JavaScript with examples.

JavaScript has two main types:

Primitive Data Types:

- String → "Hello"
- Number → 10
- Boolean → true/false
- Undefined → let x;
- Null → let y = null

Non-Primitive:

- Object → {name: "Sumit"}
- Array → [1, 2, 3]

Question 3: Difference between undefined and null

- undefined: Variable declared but not assigned a value
- null: Intentionally assigned empty value

Example:

```
let a;
```

```
let b = null;
```

## 3. JavaScript Operators

Question 1: Types of operators

Arithmetic Operators

+ - \* / %

Assignment Operators

= += -=

Comparison Operators

== === != > <

Logical Operators

&& || !

Question 2: Difference between == and ===

== → compares values only

=== → compares value + type

Example:

```
5 == "5" // true
```

```
5 === "5" // false
```

#### 4. Control Flow

Question 1: What is control flow?

Control flow determines how code executes based on conditions.

If-Else Example:

```
let age = 18;
```

```
if(age >= 18){
```

```
  console.log("Adult");
```

```
} else {
```

```
  console.log("Minor");
```

```
}
```

Question 2: Switch statement

Switch checks multiple conditions efficiently.

Example:

```
let day = 1;
```

```
switch(day){
```

```
  case 1:
```

```
    console.log("Monday");
```

```
    break;
```

```
  default:
```

```
    console.log("Invalid");
```

```
}
```

Use switch when checking one variable against multiple values.

#### 5. Loops

Question 1: Types of loops

For Loop

```
for(let i=0; i<5; i++){
```

```
  console.log(i);
```

```
}
```

While Loop

```
let i=0;
```

```
while(i<5){
```

```
  console.log(i);
```

```
  i++;
```

```
}
```

## Do-While Loop

```
let i=0;
do{
  console.log(i);
  i++;
} while(i<5);
```

### Question 2: Difference between while and do-while

- while: checks condition first
- do-while: runs at least once before checking

## 6. Functions

### Question 1: What are functions?

Functions are reusable blocks of code.

Syntax:

```
function greet(){
  console.log("Hello");
}
greet();
```

### Question 2: Function declaration vs expression

- Declaration

```
function add(a,b){
  return a+b;
}
```

- Expression

```
const add = function(a,b){
  return a+b;
}
```

### Question 3: Parameters and return values

- Parameters: inputs to a function
- Return: output from function

Example:

```
function sum(a,b){
  return a+b;
}
```

## 7. Arrays

### Question 1: What is an array?

An array stores multiple values in one variable.

```
let arr = [1,2,3];
```

#### Question 2: Array methods

- `push()` → adds element at end
- `pop()` → removes last element
- `shift()` → removes first element
- `unshift()` → adds at beginning

### 8. Objects

#### Question 1: What is an object?

Objects store data in key-value pairs.

```
let person = {  
  name: "Sumit",  
  age: 21  
};
```

Difference from arrays:

- Arrays use index
- Objects use keys

#### Question 2: Access and update properties

Dot notation

```
person.name
```

Bracket notation

```
person["name"]
```

Update:

```
person.age = 22;
```

### 9. JavaScript Events

#### Question 1: What are events?

Events are actions like clicks, typing, etc.

Event listeners allow JavaScript to respond to these actions.

#### Question 2: `addEventListener()`

Example:

```
button.addEventListener("click", function(){  
  alert("Clicked!");  
});
```

## 10. DOM Manipulation

Question 1: What is DOM?

DOM stands for Document Object Model. It represents the structure of a webpage.

JavaScript uses DOM to:

- Change content
- Modify styles
- Handle elements dynamically

Question 2: DOM selection methods

- `getElementById()` → selects by ID
- `getElementsByClassName()` → selects by class
- `querySelector()` → selects first matching element

## 11. Timing Events

Question 1: `setTimeout()` and `setInterval()`

- `setTimeout()` → runs once after delay
- `setInterval()` → runs repeatedly

Question 2: Example of `setTimeout()`

```
setTimeout(function(){  
  console.log("Hello after 2 seconds");  
}, 2000);
```

## 12. Error Handling

Question 1: `try`, `catch`, `finally`

```
try {  
  let a = b;  
} catch(error){  
  console.log("Error occurred");  
} finally {  
  console.log("Execution completed");  
}
```

Question 2: Importance of error handling

Error handling is important because:

- Prevents application crashes
- Improves user experience
- Helps in debugging
- Ensures smooth execution